SunGrid Copilot: Design Notes (Part C)

Ahya Technologies · AI Engineer assessment · Part C design write-up · github.com/ManzarAli25/sungrid-copilot

The system: section-level chunks with context headers in Qdrant; a query classifier (TypeSafe Jev, with an LLM fallback) whose output only filters retrieval; and a LangGraph agent that routes eligibility questions to the unmodified eligibility service behind a code-level hard gate. Numbers below come from `evals/` unless marked otherwise.

1. Evaluating before shipping

Offline (built). A 35-case golden set traceable to the documents: RAG per document, doc 06 from both sides, cross-category and unanswerable questions, every hard-gate failure, missing or vague inputs, multi-turn follow-ups, and off-topic questions. Two layers:

- **Deterministic checks, wherever code can decide:** category, route, retrieval hit@5, tool called or not, exact tool arguments, verdict, and "ineligible → no amount". All at 100%.
- **DeepEval LLM judges, for what code can't check:** Faithfulness 0.99, Answer Relevancy 0.96, Contextual Precision 0.95, Contextual Recall 1.00, Tool Correctness 1.00, and a G-Eval hard-gate rubric at 1.00. The judge is a different model family from the agent.

Checking the judge itself. A planted leaky answer scores 0.0 on the hard-gate rubric (a negative control). DeepEval's `llm` and `hybrid` (Jev-decided) modes disagreed by up to 0.2 on the same metric. Reading the failures showed `hybrid` explanations quoting probabilities that weren't in the answers, so `llm` is the default. Three of the first five eval findings were defects in the eval (rubric wording, golden answers, judge mode), not the product.

Repeated runs. A 7% extraction failure passed one run and failed the next. Critical behaviours (installer-approval extraction, option-order bias) were probed 30× or by reordering, and the release gate should track pass rates over N runs, not single results.

Ship gate: hard gate and tool correctness at 100% over repeated runs, no deterministic regressions, Faithfulness ≥ 0.9, and a human read of every judge failure.

Online (to add). Every turn already records category probabilities, filter, fallback, tool input/output and per-step timings. In production: log these with the answer; monitor **filter-override rate** (rising means classifier or content drift), low-margin classifications, "not in the excerpts" answers (knowledge gaps), eligibility asks that never complete, latency per step, and cost per turn. Sample live conversations for offline DeepEval scoring and human review, and add failures to the golden set.

2. What breaks first at scale

10x documents (~140 docs, ~700 chunks). Qdrant is not the bottleneck: 700 or 70k vectors with payload indexes is trivial. What breaks first is **the human-curated layer**: hand-written document→category tags and golden labels, and chunking that assumes tidy `##` sections. Changes: tag at section level, with Jev proposing tags at ingest time and people reviewing low-confidence ones; structure-aware splitting with the existing size guard for long or messy sources (PDFs); a **reranker** once near-duplicate chunks crowd the top 5; turning the explicit cross-references ("as described in the Billing FAQs") into metadata that pulls the referenced section in; and a golden set grown from sampled real queries rather than hand-written ones.

100x queries. Each question makes 4–5 external model calls (rewrite on follow-ups, two Jev decisions, an embedding, generation). What breaks first is **provider rate limits and cost**, then **in-process state**: conversation memory is in RAM (`InMemorySaver`), and the UI and agent share one Streamlit process. Changes: a persistent checkpointer (Postgres/Redis) and a stateless agent API behind the UI, scaled horizontally; both Jev questions in **one request** (it answers several questions in parallel) and running classification and the embedding concurrently (about 2 s measured headroom per turn); a semantic cache for frequent questions; skipping the rewrite when a message is already standalone; per-user rate limits; and retries with fallbacks per provider (the LLM classifier fallback already exists).

3. Adding a new document category

The design treats categories as data, so most of the system stays the same:

Changes	Stays the same
One entry in `config/categories.yaml` (name + description)	Graph, nodes, prompts, retriever
Tag new/affected documents, re-run ingestion (idempotent upserts)	Classifier code: Jev reads the YAML, **no retraining**
New golden cases; re-run evals and the strict/widened comparison	Filter mechanics (`MatchAny`) and override logic
Possibly re-tune the 0.10 widening and 0.10 override margins	Eligibility path and hard gate

A guard test fails if a document is untagged or uses an undefined category. The category description is the classifier's only definition of the category, so its wording needs testing like code. **Out of scope of "just data":** a category that needs its own tool or service (a new eligibility check) is a new intent plus a new node, which is real code.

4. Non-obvious trade-off: classifying with a decision model, not the LLM

The obvious design prompts the LLM you already use. I classify category and intent with **Jev**, a System One model that returns typed, calibrated decisions instead of text.

Gained, with the same accuracy. On 20 golden questions both backends chose the identical category every time (22/22 category accuracy each). Jev was **2.5× faster**: 0.68 s vs 1.67 s median, p90 0.73 s vs 1.99 s. Two decisions per question save about 2 s per turn. It is also **~2.8× cheaper per call**: our LLM classification uses 888 input / 30 output tokens (~$0.000104), vs ~$0.000037 for the same input at Jev's published price (estimate). Its probabilities also became a design input: the 0.84 / 0.16 split on "my rebate hasn't shown up on my bill" is the rule for widening the filter (19/22 → 21/22 retrieval hits).

Gave up: explainability. Jev returns a label and probabilities, never a reason. Calibration holds across many predictions, not for each answer, so **a confident answer can still be wrong, with no way to trace why**: "What rebate will the new multi-family tier get?" was classified `incentive_rebate` at 0.98, while the answer lives in Company Updates. An LLM classifier could at least state its reasoning for an audit. Jev also **reads literally**: it answers the question as written, so "What if I shrink it to 9 kW?" sent on its own looked like a general question, and category quality depends on the exact wording of the descriptions.

How the design absorbs it. Retrieval never trusts the label alone: if results outside the filter beat the filtered ones by ≥ 0.10, the filter is overridden (this fixed the multi-family case). Jev sees a short, purpose-built input (the standalone rewrite plus explicit context such as "the member already gave eligibility details"), and never handles numbers, which the LLM extracts and code checks. The classifier sits behind one interface with an LLM fallback (`CLASSIFIER_BACKEND`), so the project runs without a Jev key.

Out of scope (deliberately)

Authentication and PII handling for member data; persistent conversation memory; production monitoring infrastructure (the signals are captured, not shipped anywhere); a reranker; cross-reference expansion; multilingual queries; and load testing beyond single-user latency measurements.